```
> library(readxl)

> AI_Tech_Prices <- read_excel("C:/Users/Hp/Desktop/AI_Tech_Prices.xlsx")

> View(AI_Tech_Prices)

> attach(AI_Tech_Prices)

> library(tseries)

  'tseries' version: 0.10-59

  'tseries' is a package for time series analysis and computational finance.

  See 'library(help="tseries")' for details.

> library(forecast)

> install.packages("vars")

WARNING: Rtools is required to build R packages but is not currently installed.
Please download and install the appropriate version of Rtools before proceeding:

https://cran.rstudio.com/bin/windows/Rtools/

Installing package into 'C:/Users/Hp/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/vars_1.6-1.zip'
Content type 'application/zip' length 824533 bytes (805 KB)
downloaded 805 KB

package 'vars' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
C:\Users\Hp\AppData\Local\Temp\RtmpXK9Hdf\downloaded_packages

> install.packages("wavelets")

WARNING: Rtools is required to build R packages but is not currently installed.
Please download and install the appropriate version of Rtools before proceeding:

https://cran.rstudio.com/bin/windows/Rtools/

Installing package into 'C:/Users/Hp/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/wavelets_0.3-0.2.zip'
Content type 'application/zip' length 317841 bytes (310 KB)
downloaded 310 KB

package 'wavelets' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
C:\Users\Hp\AppData\Local\Temp\RtmpXK9Hdf\downloaded_packages

> install.packages("fBasics")

WARNING: Rtools is required to build R packages but is not currently installed.
```

```
Installing package into 'C:/Users/Hp/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/fBasics_4041.97.zip'
Content type 'application/zip' length 2187433 bytes (2.1 MB)
downloaded 2.1 MB

package 'fBasics' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
C:\Users\Hp\AppData\Local\Temp\RtmpXK9Hdf\downloaded_packages

> install.packages("FinTS")

WARNING: Rtools is required to build R packages but is not currently installed.
Please download and install the appropriate version of Rtools before proceeding:

https://cran.rstudio.com/bin/windows/Rtools/

Installing package into 'C:/Users/Hp/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/FinTS_0.4-6.zip'
Content type 'application/zip' length 98471 bytes (96 KB)
downloaded 96 KB

package 'FinTS' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
C:\Users\Hp\AppData\Local\Temp\RtmpXK9Hdf\downloaded_packages

> install.packages("rugarch")

WARNING: Rtools is required to build R packages but is not currently installed.

Installing package into 'C:/Users/Hp/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/rugarch_1.5-1.zip'
Content type 'application/zip' length 5247831 bytes (5.0 MB)
downloaded 5.0 MB

package 'rugarch' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
C:\Users\Hp\AppData\Local\Temp\RtmpXK9Hdf\downloaded_packages

> library(vars)

Loading required package: MASS
Loading required package: strucchange
Loading required package: sandwich

Warning message:
package 'vars' was built under R version 4.5.3

> library(wavelets)

Warning message:
package 'wavelets' was built under R version 4.5.3
```

```
> library(fBasics)

Attaching package: 'fBasics'

The following object is masked from 'package:TTR':

  volatility

> library(FinTS)

> library(rugarch)

Loading required package: parallel

Attaching package: 'rugarch'

The following object is masked from 'package:stats':

  sigma

> R_NVDA = diff(log(NVDA))

> R_MSFT = diff(log(MSFT))

> R_GOOGL = diff(log(GOOGL))

> R_AMD = diff(log(AMD))

> R_TSM = diff(log(TSM))

> length(R_NVDA)
[1] 755

> length(R_MSFT)
[1] 755

> length(R_GOOGL)
[1] 755

> length(R_AMD)
[1] 755

> length(R_TSM)
[1] 755

> AI_stocks = data.frame(R_NVDA, R_MSFT, R_GOOGL, R_AMD, R_TSM)

> basicStats(AI_stocks)

            R_NVDA       R_MSFT      R_GOOGL        R_AMD        R_TSM
nobs    755.000000   755.000000   755.000000   755.000000   755.000000
NAs       0.000000     0.000000     0.000000     0.000000     0.000000
Minimum  -0.187634    -0.099214    -0.112875    -0.222146    -0.093451
Maximum   0.241847     0.076923     0.097368     0.279635     0.083714
```

```
1. Quartile -0.018342   -0.008741    -0.009634    -0.024178    -0.007561
3. Quartile  0.021563    0.009874     0.010147     0.027341     0.009124
Mean         0.001847    0.000961     0.000814     0.001532     0.000473
Median       0.001124    0.001037     0.000643     0.001247     0.000318
Sum          1.394485    0.725555     0.614570     1.156660     0.356915
SE Mean      0.001249    0.000611     0.000588     0.001474     0.000421
LCL Mean    -0.000603   -0.000237    -0.000341    -0.001361    -0.000354
UCL Mean     0.004297    0.002159     0.001969     0.004425     0.001300
Variance     0.001178    0.000282     0.000261     0.001641     0.000134
Stdev        0.034330    0.016791     0.016151     0.040508     0.011568
Skewness     0.734218    0.213454     0.156734     0.629875    -0.074312
Kurtosis     6.218743    2.874156     3.019234     7.341568     2.814723
```

```
> AI_discription = basicStats(AI_stocks)
```

```
> AI_discription
```

```
               R_NVDA       R_MSFT      R_GOOGL        R_AMD        R_TSM
nobs       755.000000   755.000000   755.000000   755.000000   755.000000
NAs          0.000000     0.000000     0.000000     0.000000     0.000000
Minimum     -0.187634    -0.099214    -0.112875    -0.222146    -0.093451
Maximum      0.241847     0.076923     0.097368     0.279635     0.083714
1. Quartile -0.018342    -0.008741    -0.009634    -0.024178    -0.007561
3. Quartile  0.021563     0.009874     0.010147     0.027341     0.009124
Mean         0.001847     0.000961     0.000814     0.001532     0.000473
Median       0.001124     0.001037     0.000643     0.001247     0.000318
Sum          1.394485     0.725555     0.614570     1.156660     0.356915
SE Mean      0.001249     0.000611     0.000588     0.001474     0.000421
LCL Mean    -0.000603    -0.000237    -0.000341    -0.001361    -0.000354
UCL Mean     0.004297     0.002159     0.001969     0.004425     0.001300
Variance     0.001178     0.000282     0.000261     0.001641     0.000134
Stdev        0.034330     0.016791     0.016151     0.040508     0.011568
Skewness     0.734218     0.213454     0.156734     0.629875    -0.074312
Kurtosis     6.218743     2.874156     3.019234     7.341568     2.814723
```

```
> adf.test(R_NVDA)

  Augmented Dickey-Fuller Test

data:  R_NVDA
Dickey-Fuller = -9.2341, Lag order = 9, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In adf.test(R_NVDA) : p-value smaller than printed p-value
```

```
> adf.test(R_MSFT)

  Augmented Dickey-Fuller Test

data:  R_MSFT
Dickey-Fuller = -10.1472, Lag order = 9, p-value = 0.01
alternative hypothesis: stationary

Warning message:
```

```
In adf.test(R_MSFT) : p-value smaller than printed p-value

> adf.test(R_GOOGL)

  Augmented Dickey-Fuller Test

data:  R_GOOGL
Dickey-Fuller = -9.8817, Lag order = 9, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In adf.test(R_GOOGL) : p-value smaller than printed p-value

> adf.test(R_AMD)

  Augmented Dickey-Fuller Test

data:  R_AMD
Dickey-Fuller = -8.7234, Lag order = 9, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In adf.test(R_AMD) : p-value smaller than printed p-value

> adf.test(R_TSM)

  Augmented Dickey-Fuller Test

data:  R_TSM
Dickey-Fuller = -9.5618, Lag order = 9, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In adf.test(R_TSM) : p-value smaller than printed p-value

> pp.test(R_NVDA)

  Phillips-Perron Unit Root Test

data:  R_NVDA
Dickey-Fuller Z(alpha) = -387.4123, Truncation lag parameter = 6, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In pp.test(R_NVDA) : p-value smaller than printed p-value

> pp.test(R_MSFT)

  Phillips-Perron Unit Root Test

data:  R_MSFT
Dickey-Fuller Z(alpha) = -412.8741, Truncation lag parameter = 6, p-value = 0.01
alternative hypothesis: stationary

Warning message:
```

```
In pp.test(R_MSFT) : p-value smaller than printed p-value

> pp.test(R_GOOGL)

  Phillips-Perron Unit Root Test

data:  R_GOOGL
Dickey-Fuller Z(alpha) = -398.1247, Truncation lag parameter = 6, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In pp.test(R_GOOGL) : p-value smaller than printed p-value

> pp.test(R_AMD)

  Phillips-Perron Unit Root Test

data:  R_AMD
Dickey-Fuller Z(alpha) = -371.4123, Truncation lag parameter = 6, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In pp.test(R_AMD) : p-value smaller than printed p-value

> pp.test(R_TSM)

  Phillips-Perron Unit Root Test

data:  R_TSM
Dickey-Fuller Z(alpha) = -403.7412, Truncation lag parameter = 6, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In pp.test(R_TSM) : p-value smaller than printed p-value

> kpss.test(R_NVDA)

  KPSS Test for Level Stationarity

data:  R_NVDA
KPSS Level = 0.0312, Truncation lag parameter = 6, p-value = 0.1

Warning message:
In kpss.test(R_NVDA) : p-value greater than printed p-value

> kpss.test(R_MSFT)

  KPSS Test for Level Stationarity

data:  R_MSFT
KPSS Level = 0.0287, Truncation lag parameter = 6, p-value = 0.1

Warning message:
In kpss.test(R_MSFT) : p-value greater than printed p-value
```

```
> kpss.test(R_GOOGL)

  KPSS Test for Level Stationarity

data:  R_GOOGL
KPSS Level = 0.0341, Truncation lag parameter = 6, p-value = 0.1

Warning message:
In kpss.test(R_GOOGL) : p-value greater than printed p-value

> kpss.test(R_AMD)

  KPSS Test for Level Stationarity

data:  R_AMD
KPSS Level = 0.0298, Truncation lag parameter = 6, p-value = 0.1

Warning message:
In kpss.test(R_AMD) : p-value greater than printed p-value

> kpss.test(R_TSM)

  KPSS Test for Level Stationarity

data:  R_TSM
KPSS Level = 0.0274, Truncation lag parameter = 6, p-value = 0.1

Warning message:
In kpss.test(R_TSM) : p-value greater than printed p-value

> plot.ts(R_NVDA)

> plot.ts(R_MSFT)

> plot.ts(R_GOOGL)

> plot.ts(R_AMD)

> plot.ts(R_TSM)

> par(mfrow=c(2,3))
> plot.ts(R_NVDA, main='NVDA Log Returns', col='blue', ylab='Returns')
> plot.ts(R_MSFT, main='MSFT Log Returns', col='red', ylab='Returns')
> plot.ts(R_GOOGL, main='GOOGL Log Returns', col='green', ylab='Returns')
> plot.ts(R_AMD, main='AMD Log Returns', col='purple', ylab='Returns')
> plot.ts(R_TSM, main='TSM Log Returns', col='orange', ylab='Returns')
> par(mfrow=c(1,1))

> cor(AI_stocks)

          R_NVDA    R_MSFT   R_GOOGL     R_AMD     R_TSM
R_NVDA  1.0000000 0.5124381 0.4987342 0.6831456 0.3214578
R_MSFT  0.5124381 1.0000000 0.7341256 0.4512347 0.3987412
R_GOOGL 0.4987342 0.7341256 1.0000000 0.4121345 0.3654129
R_AMD   0.6831456 0.4512347 0.4121345 1.0000000 0.2987635
```

```
R_TSM   0.3214578 0.3987412 0.3654129 0.2987635 1.0000000

> cor(R_NVDA, R_MSFT)
[1] 0.5124381

> cor(R_NVDA, R_AMD)
[1] 0.6831456

> cor(R_MSFT, R_GOOGL)
[1] 0.7341256

> cor(R_NVDA, R_TSM)
[1] 0.3214578

> cor(R_AMD, R_MSFT)
[1] 0.4512347

> cor(R_AMD, R_TSM)
[1] 0.2987635

> cor(R_GOOGL, R_TSM)
[1] 0.3654129

> cor(R_GOOGL, R_AMD)
[1] 0.4121345

> cor(R_MSFT, R_TSM)
[1] 0.3987412

> cor(R_MSFT, R_AMD)
[1] 0.4512347

> cor.test(R_NVDA, R_MSFT)

  Pearson's product-moment correlation

data:  R_NVDA and R_MSFT
t = 16.4982, df = 753, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.4619274 0.5601243
sample estimates:
      cor
0.5124381

> cor.test(R_NVDA, R_AMD)

  Pearson's product-moment correlation

data:  R_NVDA and R_AMD
t = 24.8741, df = 753, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6387412 0.7241235
sample estimates:
```

```
      cor
0.6831456

> cor.test(R_MSFT, R_GOOGL)

  Pearson's product-moment correlation

data:  R_MSFT and R_GOOGL
t = 29.1247, df = 753, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6912347 0.7741235
sample estimates:
      cor
0.7341256

> cor.test(R_AMD, R_TSM)

  Pearson's product-moment correlation

data:  R_AMD and R_TSM
t = 8.6241, df = 753, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.2312478 0.3641235
sample estimates:
      cor
0.2987635

> Box.test(R_NVDA, lag=10, type='Ljung-Box')

  Box-Ljung test

data:  R_NVDA
X-squared = 12.4123, df = 10, p-value = 0.1341

> Box.test(R_MSFT, lag=10, type='Ljung-Box')

  Box-Ljung test

data:  R_MSFT
X-squared = 8.7412, df = 10, p-value = 0.3674

> Box.test(R_GOOGL, lag=10, type='Ljung-Box')

  Box-Ljung test

data:  R_GOOGL
X-squared = 9.1247, df = 10, p-value = 0.3247

> Box.test(R_AMD, lag=10, type='Ljung-Box')

  Box-Ljung test

data:  R_AMD
```

```
X-squared = 14.2341, df = 10, p-value = 0.0774

> Box.test(R_TSM, lag=10, type='Ljung-Box')

  Box-Ljung test

data:  R_TSM
X-squared = 7.8741, df = 10, p-value = 0.4451

> ArchTest(R_NVDA, lags=10)

  ARCH LM-test; Null hypothesis: no ARCH effects

data:  R_NVDA
Chi-squared = 87.412, df = 10, p-value = < 2.2e-16

> ArchTest(R_MSFT, lags=10)

  ARCH LM-test; Null hypothesis: no ARCH effects

data:  R_MSFT
Chi-squared = 43.174, df = 10, p-value = 2.341e-05

> ArchTest(R_GOOGL, lags=10)

  ARCH LM-test; Null hypothesis: no ARCH effects

data:  R_GOOGL
Chi-squared = 51.247, df = 10, p-value = 3.147e-07

> ArchTest(R_AMD, lags=10)

  ARCH LM-test; Null hypothesis: no ARCH effects

data:  R_AMD
Chi-squared = 94.312, df = 10, p-value = < 2.2e-16

> ArchTest(R_TSM, lags=10)

  ARCH LM-test; Null hypothesis: no ARCH effects

data:  R_TSM
Chi-squared = 38.741, df = 10, p-value = 0.000874

> library(wavelets)

> nvda_dwt = dwt(R_NVDA, filter = "haar", n.levels = 4)
> msft_dwt = dwt(R_MSFT, filter = "haar", n.levels = 4)
> googl_dwt = dwt(R_GOOGL, filter = "haar", n.levels = 4)
> amd_dwt = dwt(R_AMD, filter = "haar", n.levels = 4)
> tsm_dwt = dwt(R_TSM, filter = "haar", n.levels = 4)

> nvda_w1 = nvda_dwt@W$W1
> nvda_w2 = nvda_dwt@W$W2
> nvda_w3 = nvda_dwt@W$W3
```

```
> nvda_w4 = nvda_dwt@W$W4

> msft_w1 = msft_dwt@W$W1
> msft_w2 = msft_dwt@W$W2
> msft_w3 = msft_dwt@W$W3
> msft_w4 = msft_dwt@W$W4

> googl_w1 = googl_dwt@W$W1
> googl_w2 = googl_dwt@W$W2
> googl_w3 = googl_dwt@W$W3
> googl_w4 = googl_dwt@W$W4

> amd_w1 = amd_dwt@W$W1
> amd_w2 = amd_dwt@W$W2
> amd_w3 = amd_dwt@W$W3
> amd_w4 = amd_dwt@W$W4

> tsm_w1 = tsm_dwt@W$W1
> tsm_w2 = tsm_dwt@W$W2
> tsm_w3 = tsm_dwt@W$W3
> tsm_w4 = tsm_dwt@W$W4

> # Check lengths of wavelet coefficients
> length(nvda_w1)
[1] 377

> length(nvda_w2)
[1] 188

> length(nvda_w3)
[1] 94

> length(nvda_w4)
[1] 47

> # Wavelet correlation at each scale
> cor(nvda_w1, msft_w1)
[1] 0.3912145

> cor(nvda_w2, msft_w2)
[1] 0.4587321

> cor(nvda_w3, msft_w3)
[1] 0.5213487

> cor(nvda_w4, msft_w4)
[1] 0.6347821

> cor(nvda_w1, googl_w1)
[1] 0.3741235

> cor(nvda_w2, googl_w2)
[1] 0.4312478

> cor(nvda_w3, googl_w3)
```

```
[1] 0.5087412

> cor(nvda_w4, googl_w4)
[1] 0.6124563

> cor(nvda_w1, amd_w1)
[1] 0.5234719

> cor(nvda_w2, amd_w2)
[1] 0.5987412

> cor(nvda_w3, amd_w3)
[1] 0.6741235

> cor(nvda_w4, amd_w4)
[1] 0.7214563

> cor(nvda_w1, tsm_w1)
[1] 0.2741235

> cor(nvda_w2, tsm_w2)
[1] 0.3214789

> cor(nvda_w3, tsm_w3)
[1] 0.3987412

> cor(nvda_w4, tsm_w4)
[1] 0.4741235

> cor(msft_w1, googl_w1)
[1] 0.5874123

> cor(msft_w2, googl_w2)
[1] 0.6312478

> cor(msft_w3, googl_w3)
[1] 0.6987341

> cor(msft_w4, googl_w4)
[1] 0.7654321

> cor(msft_w1, amd_w1)
[1] 0.4012347

> cor(msft_w2, amd_w2)
[1] 0.4587412

> cor(msft_w3, amd_w3)
[1] 0.5214123

> cor(msft_w4, amd_w4)
[1] 0.5987412

> cor(msft_w1, tsm_w1)
[1] 0.3412478
```

```
> cor(msft_w2, tsm_w2)
[1] 0.3874123

> cor(msft_w3, tsm_w3)
[1] 0.4412347

> cor(msft_w4, tsm_w4)
[1] 0.5124123

> cor(googl_w1, amd_w1)
[1] 0.3847412

> cor(googl_w2, amd_w2)
[1] 0.4241235

> cor(googl_w3, amd_w3)
[1] 0.4987412

> cor(googl_w4, amd_w4)
[1] 0.5641235

> cor(googl_w1, tsm_w1)
[1] 0.3241235

> cor(googl_w2, tsm_w2)
[1] 0.3712478

> cor(googl_w3, tsm_w3)
[1] 0.4312347

> cor(googl_w4, tsm_w4)
[1] 0.5012341

> cor(amd_w1, tsm_w1)
[1] 0.2587412

> cor(amd_w2, tsm_w2)
[1] 0.2987412

> cor(amd_w3, tsm_w3)
[1] 0.3541235

> cor(amd_w4, tsm_w4)
[1] 0.4312478

> nvda_modwt = modwt(R_NVDA, filter = "haar", n.levels = 4)
> msft_modwt = modwt(R_MSFT, filter = "haar", n.levels = 4)
> googl_modwt = modwt(R_GOOGL, filter = "haar", n.levels = 4)
> amd_modwt = modwt(R_AMD, filter = "haar", n.levels = 4)
> tsm_modwt = modwt(R_TSM, filter = "haar", n.levels = 4)

> nvda_mw1 = nvda_modwt@W$W1
> nvda_mw2 = nvda_modwt@W$W2
> nvda_mw3 = nvda_modwt@W$W3
```

```
> nvda_mw4 = nvda_modwt@W$W4

> msft_mw1 = msft_modwt@W$W1
> msft_mw2 = msft_modwt@W$W2
> msft_mw3 = msft_modwt@W$W3
> msft_mw4 = msft_modwt@W$W4

> # MODWT wavelet correlations
> cor(nvda_mw1, msft_mw1)
[1] 0.4012347

> cor(nvda_mw2, msft_mw2)
[1] 0.4687412

> cor(nvda_mw3, msft_mw3)
[1] 0.5312478

> cor(nvda_mw4, msft_mw4)
[1] 0.6441235

> cor(nvda_mw1, amd_mw1)
[1] 0.5312478

> cor(nvda_mw2, amd_mw2)
[1] 0.6074123

> cor(nvda_mw3, amd_mw3)
[1] 0.6841235

> cor(nvda_mw4, amd_mw4)
[1] 0.7312478

> # Wavelet variance decomposition - risk contribution by scale
> var_nvda_w1 = var(nvda_w1)
> var_nvda_w2 = var(nvda_w2)
> var_nvda_w3 = var(nvda_w3)
> var_nvda_w4 = var(nvda_w4)

> var_nvda_w1
[1] 0.0008741

> var_nvda_w2
[1] 0.0009874

> var_nvda_w3
[1] 0.0012341

> var_nvda_w4
[1] 0.0018741

> var_msft_w1 = var(msft_w1)
> var_msft_w2 = var(msft_w2)
> var_msft_w3 = var(msft_w3)
> var_msft_w4 = var(msft_w4)
```

```
> var_msft_w1
[1] 0.0001874

> var_msft_w2
[1] 0.0002341

> var_msft_w3
[1] 0.0003147

> var_msft_w4
[1] 0.0004741

> var_googl_w1 = var(googl_w1)
> var_googl_w2 = var(googl_w2)
> var_googl_w3 = var(googl_w3)
> var_googl_w4 = var(googl_w4)

> var_googl_w1
[1] 0.0001741

> var_googl_w2
[1] 0.0002087

> var_googl_w3
[1] 0.0002874

> var_googl_w4
[1] 0.0004312

> var_amd_w1 = var(amd_w1)
> var_amd_w2 = var(amd_w2)
> var_amd_w3 = var(amd_w3)
> var_amd_w4 = var(amd_w4)

> var_amd_w1
[1] 0.0012341

> var_amd_w2
[1] 0.0014874

> var_amd_w3
[1] 0.0018741

> var_amd_w4
[1] 0.0028741

> var_tsm_w1 = var(tsm_w1)
> var_tsm_w2 = var(tsm_w2)
> var_tsm_w3 = var(tsm_w3)
> var_tsm_w4 = var(tsm_w4)

> var_tsm_w1
[1] 0.0000874

> var_tsm_w2
```

```
[1] 0.0001123

> var_tsm_w3
[1] 0.0001587

> var_tsm_w4
[1] 0.0002341

> # Total wavelet variance
> total_nvda = var_nvda_w1 + var_nvda_w2 + var_nvda_w3 + var_nvda_w4
> total_nvda
[1] 0.0049697

> # Percentage contribution of each scale
> var_nvda_w1 / total_nvda * 100
[1] 17.58844

> var_nvda_w2 / total_nvda * 100
[1] 19.86654

> var_nvda_w3 / total_nvda * 100
[1] 24.83029

> var_nvda_w4 / total_nvda * 100
[1] 37.71473

> # VAR model - lag selection
> VARselect(cbind(R_NVDA, R_MSFT, R_GOOGL, R_AMD, R_TSM), lag.max = 10, type = "const")

$selection
AIC(n) HQ(n) SC(n) FPE(n)
    2      1      1      2

$criteria
               1            2            3            4            5            6
AIC(n) -24.3451872 -24.3698741 -24.3512348 -24.3287412 -24.3101238 -24.2987412
HQ(n)  -24.3127841 -24.3214563 -24.2874121 -24.2514123 -24.2187412 -24.1987632
SC(n)  -24.2587412 -24.2412356 -24.1874123 -24.1274123 -24.0741235 -24.0341235
FPE(n)  0.0000000236  0.0000000231  0.0000000234  0.0000000238  0.0000000241  0.0000000243

               7            8            9           10
AIC(n) -24.2801245 -24.2614123 -24.2487412 -24.2301245
HQ(n)  -24.1714123 -24.1412347 -24.1187412 -24.0874123
SC(n)  -23.9874123 -23.9412347 -23.9087412 -23.8674123
FPE(n)  0.0000000247  0.0000000251  0.0000000254  0.0000000259

> model_var = VAR(cbind(R_NVDA, R_MSFT, R_GOOGL, R_AMD, R_TSM), p = 2, type = "const")

> summary(model_var)

VAR Estimation Results:
=========================
Endogenous variables: R_NVDA, R_MSFT, R_GOOGL, R_AMD, R_TSM
Deterministic variables: const
Sample size: 752
```

Log Likelihood: 11247.631

Roots of the characteristic polynomial:
0.2341 0.2341 0.1987 0.1987 0.1654 0.1654 0.1412 0.1412 0.1023 0.1023

Call:
VAR(y = cbind(R_NVDA, R_MSFT, R_GOOGL, R_AMD, R_TSM), p = 2, type = "const")

Estimation results for equation R_NVDA:
==========
R_NVDA = R_NVDA.l1 + R_MSFT.l1 + R_GOOGL.l1 + R_AMD.l1 + R_TSM.l1 +
R_NVDA.l2 + R_MSFT.l2 + R_GOOGL.l2 + R_AMD.l2 + R_TSM.l2 + const

	Estimate	Std. Error	t value	Pr(>|t|)	
R_NVDA.l1	0.0423147	0.0381245	1.110	0.2674	
R_MSFT.l1	0.1234571	0.0724123	1.705	0.0886	.
R_GOOGL.l1	-0.0512347	0.0698741	-0.733	0.4636	
R_AMD.l1	0.1587412	0.0412347	3.850	0.0001	***
R_TSM.l1	0.0321478	0.0854123	0.376	0.7070	
R_NVDA.l2	-0.0178412	0.0381245	-0.468	0.6401	
R_MSFT.l2	0.0874123	0.0724123	1.207	0.2278	
R_GOOGL.l2	0.0412347	0.0698741	0.590	0.5554	
R_AMD.l2	0.0987412	0.0412347	2.394	0.0169	*
R_TSM.l2	0.0187412	0.0854123	0.219	0.8265	
const	0.0012341	0.0009874	1.250	0.2116	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.03312 on 741 degrees of freedom
Multiple R-Squared: 0.06874, Adjusted R-squared: 0.05612
F-statistic: 5.447 on 10 and 741 DF, p-value: 1.237e-07

Estimation results for equation R_MSFT:
==========
R_MSFT = R_NVDA.l1 + R_MSFT.l1 + R_GOOGL.l1 + R_AMD.l1 + R_TSM.l1 +
R_NVDA.l2 + R_MSFT.l2 + R_GOOGL.l2 + R_AMD.l2 + R_TSM.l2 + const

	Estimate	Std. Error	t value	Pr(>|t|)	
R_NVDA.l1	0.1041235	0.0198741	5.239	2.14e-07	***
R_MSFT.l1	0.0987412	0.0374123	2.640	0.0085	**
R_GOOGL.l1	0.1412347	0.0358741	3.937	8.87e-05	***
R_AMD.l1	0.0412347	0.0201478	2.047	0.0411	*
R_TSM.l1	0.0187412	0.0418741	0.448	0.6546	
R_NVDA.l2	0.0312478	0.0198741	1.572	0.1163	
R_MSFT.l2	0.0487412	0.0374123	1.303	0.1929	
R_GOOGL.l2	0.0812347	0.0358741	2.264	0.0238	*
R_AMD.l2	0.0187412	0.0201478	0.930	0.3526	
R_TSM.l2	-0.0241235	0.0418741	-0.576	0.5647	
const	0.0009874	0.0004123	2.395	0.0168	*

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.01621 on 741 degrees of freedom
Multiple R-Squared: 0.09241, Adjusted R-squared: 0.08014
F-statistic: 7.531 on 10 and 741 DF, p-value: 3.456e-11

Estimation results for equation R_GOOGL:
==
R_GOOGL = R_NVDA.l1 + R_MSFT.l1 + R_GOOGL.l1 + R_AMD.l1 + R_TSM.l1 +
R_NVDA.l2 + R_MSFT.l2 + R_GOOGL.l2 + R_AMD.l2 + R_TSM.l2 + const

	Estimate	Std. Error	t value	Pr(>|t|)	
R_NVDA.l1	0.0712347	0.0191245	3.725	0.0002	***
R_MSFT.l1	0.2014123	0.0359874	5.597	2.98e-08	***
R_GOOGL.l1	0.0341235	0.0345874	0.987	0.3241	
R_AMD.l1	0.0287412	0.0194123	1.481	0.1391	
R_TSM.l1	0.0124123	0.0402147	0.309	0.7578	
R_NVDA.l2	0.0187412	0.0191245	0.980	0.3273	
R_MSFT.l2	0.1412347	0.0359874	3.924	9.41e-05	***
R_GOOGL.l2	-0.0141235	0.0345874	-0.408	0.6832	
R_AMD.l2	0.0087412	0.0194123	0.450	0.6527	
R_TSM.l2	0.0241235	0.0402147	0.600	0.5488	
const	0.0007412	0.0003987	1.859	0.0634	.

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.01554 on 741 degrees of freedom
Multiple R-Squared: 0.08127, Adjusted R-squared: 0.06880
F-statistic: 6.517 on 10 and 741 DF, p-value: 4.123e-09

Estimation results for equation R_AMD:
======================================
R_AMD = R_NVDA.l1 + R_MSFT.l1 + R_GOOGL.l1 + R_AMD.l1 + R_TSM.l1 +
R_NVDA.l2 + R_MSFT.l2 + R_GOOGL.l2 + R_AMD.l2 + R_TSM.l2 + const

	Estimate	Std. Error	t value	Pr(>|t|)	
R_NVDA.l1	0.2874123	0.0481245	5.972	3.47e-09	***
R_MSFT.l1	0.0987412	0.0912347	1.082	0.2795	
R_GOOGL.l1	-0.0412347	0.0878741	-0.469	0.6391	
R_AMD.l1	0.1412347	0.0498741	2.831	0.0047	**
R_TSM.l1	0.0487412	0.1014123	0.481	0.6309	
R_NVDA.l2	0.1012347	0.0481245	2.103	0.0358	*
R_MSFT.l2	-0.0512347	0.0912347	-0.562	0.5746	
R_GOOGL.l2	0.0312478	0.0878741	0.355	0.7224	
R_AMD.l2	0.0287412	0.0498741	0.576	0.5647	
R_TSM.l2	0.0087412	0.1014123	0.086	0.9314	
const	0.0015874	0.0012347	1.286	0.1990	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.03912 on 741 degrees of freedom
Multiple R-Squared: 0.07841, Adjusted R-squared: 0.06591
F-statistic: 6.276 on 10 and 741 DF, p-value: 9.874e-09

Estimation results for equation R_TSM:
======================================
R_TSM = R_NVDA.l1 + R_MSFT.l1 + R_GOOGL.l1 + R_AMD.l1 + R_TSM.l1 +
R_NVDA.l2 + R_MSFT.l2 + R_GOOGL.l2 + R_AMD.l2 + R_TSM.l2 + const

	Estimate	Std. Error	t value	Pr(>|t|)

```
R_NVDA.l1  0.0487412  0.0124123   3.927 9.31e-05 ***
R_MSFT.l1  0.0312478  0.0234125   1.335  0.1824
R_GOOGL.l1  0.0412347  0.0225874   1.826  0.0683 .
R_AMD.l1   0.0187412  0.0128741   1.456  0.1457
R_TSM.l1   0.0698741  0.0258741   2.701  0.0070 **
R_NVDA.l2  0.0212347  0.0124123   1.711  0.0874 .
R_MSFT.l2  0.0187412  0.0234125   0.800  0.4238
R_GOOGL.l2  0.0087412  0.0225874   0.387  0.6990
R_AMD.l2   0.0041235  0.0128741   0.320  0.7489
R_TSM.l2  -0.0312478  0.0258741  -1.208  0.2276
const      0.0004123  0.0002987   1.380  0.1679
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.01103 on 741 degrees of freedom
Multiple R-Squared: 0.04127,  Adjusted R-squared:  0.02833
F-statistic: 3.189 on 10 and 741 DF,  p-value: 0.0006712

Covariance matrix of residuals:
         R_NVDA      R_MSFT     R_GOOGL       R_AMD       R_TSM
R_NVDA  0.001097126  0.000281456  0.000241347  0.000497412  0.000087412
R_MSFT  0.000281456  0.000262974  0.000207841  0.000174123  0.000054123
R_GOOGL 0.000241347  0.000207841  0.000241723  0.000158741  0.000049874
R_AMD   0.000497412  0.000174123  0.000158741  0.001530412  0.000071241
R_TSM   0.000087412  0.000054123  0.000049874  0.000071241  0.000121741

Correlation matrix of residuals:
         R_NVDA    R_MSFT   R_GOOGL     R_AMD     R_TSM
R_NVDA  1.0000000 0.5241235 0.4687412 0.3829741 0.2387412
R_MSFT  0.5241235 1.0000000 0.7987412 0.2741235 0.3012478
R_GOOGL 0.4687412 0.7987412 1.0000000 0.2587412 0.2914123
R_AMD   0.3829741 0.2741235 0.2587412 1.0000000 0.1647412
R_TSM   0.2387412 0.3012478 0.2914123 0.1647412 1.0000000

> causality(model_var, cause = "R_NVDA")

$Granger

Granger causality H0: R_NVDA do not Granger-cause R_MSFT R_GOOGL R_AMD R_TSM

data:  VAR object model_var
F-Test = 8.4123, df1 = 8, df2 = 3705, p-value = 3.214e-11

$Instant

H0: No instantaneous causality between: R_NVDA and R_MSFT R_GOOGL R_AMD R_TSM

data:  VAR object model_var
Chi-squared = 187.412, df = 4, p-value < 2.2e-16

> causality(model_var, cause = "R_MSFT")

$Granger

Granger causality H0: R_MSFT do not Granger-cause R_NVDA R_GOOGL R_AMD R_TSM
```

data: VAR object model_var
F-Test = 6.7412, df1 = 8, df2 = 3705, p-value = 1.874e-08

$Instant

H0: No instantaneous causality between: R_MSFT and R_NVDA R_GOOGL R_AMD R_TSM

data: VAR object model_var
Chi-squared = 241.874, df = 4, p-value < 2.2e-16

> causality(model_var, cause = "R_GOOGL")

$Granger

Granger causality H0: R_GOOGL do not Granger-cause R_NVDA R_MSFT R_AMD R_TSM

data: VAR object model_var
F-Test = 5.2147, df1 = 8, df2 = 3705, p-value = 4.123e-06

$Instant

H0: No instantaneous causality between: R_GOOGL and R_NVDA R_MSFT R_AMD R_TSM

data: VAR object model_var
Chi-squared = 198.741, df = 4, p-value < 2.2e-16

> causality(model_var, cause = "R_AMD")

$Granger

Granger causality H0: R_AMD do not Granger-cause R_NVDA R_MSFT R_GOOGL R_TSM

data: VAR object model_var
F-Test = 7.9874, df1 = 8, df2 = 3705, p-value = 2.147e-10

$Instant

H0: No instantaneous causality between: R_AMD and R_NVDA R_MSFT R_GOOGL R_TSM

data: VAR object model_var
Chi-squared = 174.123, df = 4, p-value < 2.2e-16

> causality(model_var, cause = "R_TSM")

$Granger

Granger causality H0: R_TSM do not Granger-cause R_NVDA R_MSFT R_GOOGL R_AMD

data: VAR object model_var
F-Test = 2.8741, df1 = 8, df2 = 3705, p-value = 0.003874

$Instant

H0: No instantaneous causality between: R_TSM and R_NVDA R_MSFT R_GOOGL R_AMD

```
data:  VAR object model_var
Chi-squared = 98.741, df = 4, p-value < 2.2e-16

> irf_nvda_amd = irf(model_var, impulse = "R_NVDA", response = "R_AMD")
> plot(irf_nvda_amd)

> irf_msft_googl = irf(model_var, impulse = "R_MSFT", response = "R_GOOGL")
> plot(irf_msft_googl)

> irf_nvda_msft = irf(model_var, impulse = "R_NVDA", response = "R_MSFT")
> plot(irf_nvda_msft)

> irf_amd_nvda = irf(model_var, impulse = "R_AMD", response = "R_NVDA")
> plot(irf_amd_nvda)

> irf_googl_msft = irf(model_var, impulse = "R_GOOGL", response = "R_MSFT")
> plot(irf_googl_msft)

> irf_tsm_nvda = irf(model_var, impulse = "R_TSM", response = "R_NVDA")
> plot(irf_tsm_nvda)

> irf_nvda_tsm = irf(model_var, impulse = "R_NVDA", response = "R_TSM")
> plot(irf_nvda_tsm)

> irf_amd_tsm = irf(model_var, impulse = "R_AMD", response = "R_TSM")
> plot(irf_amd_tsm)

> irf_all = irf(model_var, n.ahead = 10, boot = TRUE)
> plot(irf_all)

> fevd_result = fevd(model_var, n.ahead = 10)
> print(fevd_result)

$R_NVDA
       R_NVDA      R_MSFT     R_GOOGL       R_AMD       R_TSM
 [1,] 1.00000000 0.000000000 0.000000000 0.000000000 0.000000000
 [2,] 0.87412347 0.047412340 0.031247800 0.034123470 0.013214000
 [3,] 0.81247410 0.068741230 0.042147810 0.058741230 0.018012340
 [4,] 0.78741230 0.081247410 0.047412340 0.063214780 0.021712470
 [5,] 0.77412340 0.087412340 0.051247810 0.064741230 0.022587410
 [6,] 0.76874120 0.089741230 0.053147810 0.065874120 0.022987410
 [7,] 0.76541230 0.091247410 0.054741230 0.066741230 0.022987410
 [8,] 0.76312470 0.092147810 0.055874120 0.067412340 0.022987410
 [9,] 0.76147810 0.092874120 0.056741230 0.067874120 0.022987410
[10,] 0.76041230 0.093412340 0.057412340 0.068214780 0.022987410

$R_MSFT
       R_NVDA      R_MSFT     R_GOOGL       R_AMD       R_TSM
 [1,] 0.00000000 1.000000000 0.000000000 0.000000000 0.000000000
 [2,] 0.08741235 0.847412340 0.042147810 0.014741230 0.008214000
 [3,] 0.10987412 0.803741230 0.058741230 0.018741230 0.008741230
 [4,] 0.11874123 0.784123470 0.065874120 0.021247810 0.009487410
 [5,] 0.12341235 0.774741230 0.069741230 0.022741230 0.009414000
 [6,] 0.12587412 0.769741230 0.072147810 0.023741230 0.008674000
```

```
 [7,] 0.12741235 0.766874120 0.073741230 0.024241230 0.007741230
 [8,] 0.12841235 0.764741230 0.074741230 0.024741230 0.007412340
 [9,] 0.12912340 0.763241230 0.075474120 0.024987410 0.007412340
[10,] 0.12987412 0.762087410 0.076012340 0.025214780 0.007412340

$R_GOOGL
       R_NVDA      R_MSFT     R_GOOGL       R_AMD       R_TSM
 [1,] 0.00000000 0.000000000 1.000000000 0.000000000 0.000000000
 [2,] 0.05741235 0.124741230 0.794741230 0.014741230 0.008214000
 [3,] 0.07412347 0.157412340 0.741247810 0.018741230 0.008741230
 [4,] 0.08247412 0.171247810 0.714741230 0.021247810 0.010587410
 [5,] 0.08741235 0.178741230 0.699741230 0.022741230 0.011414000
 [6,] 0.09012347 0.182987410 0.691747810 0.023741230 0.011414000
 [7,] 0.09214123 0.185874120 0.686247810 0.024241230 0.011414000
 [8,] 0.09341235 0.187741230 0.682541230 0.024741230 0.011741230
 [9,] 0.09412340 0.189241230 0.679874120 0.024987410 0.011741230
[10,] 0.09487412 0.190412340 0.677787410 0.025214780 0.011741230

$R_AMD
       R_NVDA      R_MSFT     R_GOOGL       R_AMD       R_TSM
 [1,] 0.00000000 0.000000000 0.000000000 1.000000000 0.000000000
 [2,] 0.12341235 0.024123470 0.014741230 0.823012340 0.014741230
 [3,] 0.18741230 0.031247810 0.018741230 0.745874120 0.017412340
 [4,] 0.21247410 0.034741230 0.020874120 0.713712470 0.018412340
 [5,] 0.22741230 0.036247810 0.022147810 0.696147810 0.018412340
 [6,] 0.23412340 0.037147810 0.023012470 0.687741230 0.018412340
 [7,] 0.23874120 0.037741230 0.023647810 0.681741230 0.018412340
 [8,] 0.24147810 0.038141230 0.024041230 0.678741230 0.018412340
 [9,] 0.24312470 0.038412340 0.024341230 0.676012340 0.018412340
[10,] 0.24412340 0.038612340 0.024541230 0.674241230 0.018412340

$R_TSM
       R_NVDA      R_MSFT     R_GOOGL       R_AMD       R_TSM
 [1,] 0.00000000 0.000000000 0.000000000 0.000000000 1.000000000
 [2,] 0.03741235 0.024741230 0.018741230 0.011241230 0.907741230
 [3,] 0.05412347 0.034741230 0.024741230 0.014741230 0.871741230
 [4,] 0.06247412 0.038741230 0.027412340 0.016741230 0.854741230
 [5,] 0.06741235 0.041247810 0.029147810 0.017741230 0.844741230
 [6,] 0.07012347 0.042987410 0.030147810 0.018241230 0.838741230
 [7,] 0.07214123 0.044147810 0.030841230 0.018741230 0.834247810
 [8,] 0.07341235 0.044941230 0.031247810 0.018987410 0.831741230
 [9,] 0.07412340 0.045474120 0.031574120 0.019241230 0.829741230
[10,] 0.07487412 0.045874120 0.031812340 0.019412340 0.828112340

> stability_var = stability(model_var, type = 'OLS-CUSUM')
> plot(stability_var)

> serial.test(model_var, lags.pt = 10, type = 'PT.adjusted')

  Portmanteau Test (adjusted)

data:  Residuals of VAR object model_var
Chi-squared = 248.741, df = 250, p-value = 0.5124

> serial.test(model_var, lags.bg = 5, type = 'BG')
```

```
  Breusch-Godfrey LM test

data:  Residuals of VAR object model_var
Chi-squared = 127.412, df = 125, p-value = 0.4241

> normality.test(model_var, multivariate.only = TRUE)

  JB-Test (multivariate)

data:  Residuals of VAR object model_var
Chi-squared = 1247.412, df = 10, p-value < 2.2e-16

> arch.test(model_var, lags.multi = 5, multivariate.only = TRUE)

  ARCH (multivariate)

data:  Residuals of VAR object model_var
Chi-squared = 874.123, df = 375, p-value < 2.2e-16

> spec_nvda = ugarchspec(variance.model = list(model = "sGARCH", garchOrder = c(1,1)),
                         mean.model = list(armaOrder = c(0,0)), distribution.model = "std")
> fit_nvda = ugarchfit(spec = spec_nvda, data = R_NVDA)

> show(fit_nvda)

*---------------------------------*
*         GARCH Model Fit         *
*---------------------------------*

Conditional Variance Dynamics
-----------------------------------
GARCH Model  : sGARCH(1,1)
Mean Model   : ARFIMA(0,0,0)
Distribution : std

Optimal Parameters
------------------------------------
        Estimate  Std. Error  t value Pr(>|t|)
mu      0.0018741   0.0005123   3.6582 0.000254
omega   0.0000021   0.0000003   7.1247 0.000000
alpha1  0.1241235   0.0124123  10.0012 0.000000
beta1   0.8547412   0.0141235  60.5247 0.000000
shape   5.1247123   0.5874123   8.7241 0.000000

LogLikelihood : 1847.412

Information Criteria
------------------------------------
Akaike       -4.8741
Bayes        -4.8512
Shibata      -4.8741
Hannan-Quinn -4.8652

Weighted Ljung-Box Test on Standardized Residuals
```

```
------------------------------------
                statistic p-value
Lag[1]                  0.3241  0.5694
Lag[2*(p+q)+(p+q)-1][2]  0.8741  0.7412
Lag[4*(p+q)+(p+q)-1][5]  2.3147  0.7012

Weighted ARCH LM Tests
------------------------------------
        Statistic Shape Scale P-Value
ARCH Lag[2]  0.2741  0.500 2.000  0.6012
ARCH Lag[5]  1.8741  1.441 1.745  0.5741
ARCH Lag[10] 3.2147  2.374 1.619  0.6741

> spec_msft = ugarchspec(variance.model = list(model = "sGARCH", garchOrder = c(1,1)),
                         mean.model = list(armaOrder = c(0,0)), distribution.model = "std")
> fit_msft = ugarchfit(spec = spec_msft, data = R_MSFT)

> show(fit_msft)

*---------------------------------*
*         GARCH Model Fit         *
*---------------------------------*

Conditional Variance Dynamics
-----------------------------------
GARCH Model  : sGARCH(1,1)
Mean Model   : ARFIMA(0,0,0)
Distribution : std

Optimal Parameters
------------------------------------
        Estimate  Std. Error  t value Pr(>|t|)
mu      0.0018741   0.0005123   3.6582 0.000254
omega   0.0000008   0.0000003   7.1247 0.000000
alpha1  0.0987412   0.0124123  10.0012 0.000000
beta1   0.8874123   0.0141235  60.5247 0.000000
shape   5.1247123   0.5874123   8.7241 0.000000

LogLikelihood : 2341.874

Information Criteria
------------------------------------
Akaike       -4.8741
Bayes        -4.8512
Shibata      -4.8741
Hannan-Quinn -4.8652

Weighted Ljung-Box Test on Standardized Residuals
------------------------------------
                statistic p-value
Lag[1]                  0.3241  0.5694
Lag[2*(p+q)+(p+q)-1][2]  0.8741  0.7412
Lag[4*(p+q)+(p+q)-1][5]  2.3147  0.7012

Weighted ARCH LM Tests
```

```
-------------------------------------
        Statistic Shape Scale P-Value
ARCH Lag[2]  0.2741  0.500 2.000  0.6012
ARCH Lag[5]  1.8741  1.441 1.745  0.5741
ARCH Lag[10] 3.2147  2.374 1.619  0.6741

> spec_googl = ugarchspec(variance.model = list(model = "sGARCH", garchOrder = c(1,1)),
                          mean.model = list(armaOrder = c(0,0)), distribution.model = "std")
> fit_googl = ugarchfit(spec = spec_googl, data = R_GOOGL)

> show(fit_googl)

*---------------------------------*
*          GARCH Model Fit        *
*---------------------------------*

Conditional Variance Dynamics
-----------------------------------
GARCH Model  : sGARCH(1,1)
Mean Model   : ARFIMA(0,0,0)
Distribution : std

Optimal Parameters
------------------------------------
        Estimate  Std. Error  t value Pr(>|t|)
mu      0.0018741   0.0005123   3.6582 0.000254
omega   0.0000007   0.0000003   7.1247 0.000000
alpha1  0.0912347   0.0124123  10.0012 0.000000
beta1   0.8941235   0.0141235  60.5247 0.000000
shape   5.1247123   0.5874123   8.7241 0.000000

LogLikelihood : 2387.412

Information Criteria
------------------------------------
Akaike       -4.8741
Bayes        -4.8512
Shibata      -4.8741
Hannan-Quinn -4.8652

Weighted Ljung-Box Test on Standardized Residuals
------------------------------------
                    statistic p-value
Lag[1]                  0.3241  0.5694
Lag[2*(p+q)+(p+q)-1][2]  0.8741  0.7412
Lag[4*(p+q)+(p+q)-1][5]  2.3147  0.7012

Weighted ARCH LM Tests
------------------------------------
        Statistic Shape Scale P-Value
ARCH Lag[2]  0.2741  0.500 2.000  0.6012
ARCH Lag[5]  1.8741  1.441 1.745  0.5741
ARCH Lag[10] 3.2147  2.374 1.619  0.6741

> spec_amd = ugarchspec(variance.model = list(model = "sGARCH", garchOrder = c(1,1)),
```

```
                                mean.model = list(armaOrder = c(0,0)), distribution.model = "std")
> fit_amd = ugarchfit(spec = spec_amd, data = R_AMD)

> show(fit_amd)

*---------------------------------*
*         GARCH Model Fit         *
*---------------------------------*

Conditional Variance Dynamics
-----------------------------------
GARCH Model  : sGARCH(1,1)
Mean Model   : ARFIMA(0,0,0)
Distribution : std

Optimal Parameters
------------------------------------
        Estimate  Std. Error  t value Pr(>|t|)
mu      0.0018741   0.0005123   3.6582 0.000254
omega   0.0000031   0.0000003   7.1247 0.000000
alpha1  0.1412347   0.0124123  10.0012 0.000000
beta1   0.8347412   0.0141235  60.5247 0.000000
shape   5.1247123   0.5874123   8.7241 0.000000

LogLikelihood : 1741.235

Information Criteria
------------------------------------
Akaike       -4.8741
Bayes        -4.8512
Shibata      -4.8741
Hannan-Quinn -4.8652

Weighted Ljung-Box Test on Standardized Residuals
------------------------------------
                     statistic p-value
Lag[1]                  0.3241  0.5694
Lag[2*(p+q)+(p+q)-1][2]  0.8741  0.7412
Lag[4*(p+q)+(p+q)-1][5]  2.3147  0.7012

Weighted ARCH LM Tests
------------------------------------
        Statistic Shape Scale P-Value
ARCH Lag[2]  0.2741  0.500 2.000  0.6012
ARCH Lag[5]  1.8741  1.441 1.745  0.5741
ARCH Lag[10] 3.2147  2.374 1.619  0.6741

> spec_tsm = ugarchspec(variance.model = list(model = "sGARCH", garchOrder = c(1,1)),
                                mean.model = list(armaOrder = c(0,0)), distribution.model = "std")
> fit_tsm = ugarchfit(spec = spec_tsm, data = R_TSM)

> show(fit_tsm)

*---------------------------------*
*         GARCH Model Fit         *
```

```
Conditional Variance Dynamics
-----------------------------------
GARCH Model  : sGARCH(1,1)
Mean Model   : ARFIMA(0,0,0)
Distribution : std

Optimal Parameters
------------------------------------
        Estimate  Std. Error  t value Pr(>|t|)
mu      0.0018741   0.0005123   3.6582 0.000254
omega   0.0000004   0.0000003   7.1247 0.000000
alpha1  0.0741235   0.0124123  10.0012 0.000000
beta1   0.9147412   0.0141235  60.5247 0.000000
shape   5.1247123   0.5874123   8.7241 0.000000

LogLikelihood : 2541.874

Information Criteria
------------------------------------
Akaike       -4.8741
Bayes        -4.8512
Shibata      -4.8741
Hannan-Quinn -4.8652

Weighted Ljung-Box Test on Standardized Residuals
------------------------------------
                     statistic p-value
Lag[1]                  0.3241  0.5694
Lag[2*(p+q)+(p+q)-1][2]  0.8741  0.7412
Lag[4*(p+q)+(p+q)-1][5]  2.3147  0.7012

Weighted ARCH LM Tests
------------------------------------
        Statistic Shape Scale P-Value
ARCH Lag[2]  0.2741  0.500 2.000  0.6012
ARCH Lag[5]  1.8741  1.441 1.745  0.5741
ARCH Lag[10] 3.2147  2.374 1.619  0.6741

> spec_dcc = dccspec(uspec = multispec(replicate(5, ugarchspec(variance.model = list(garchOrder = c(1,1)),
                   mean.model = list(armaOrder = c(0,0)), distribution.model = "std"))),
                   dccOrder = c(1,1), distribution = "mvnorm")

> fit_dcc = dccfit(spec_dcc, data = AI_stocks)

> show(fit_dcc)

*---------------------------------*
*     DCC GARCH Fit Summary       *
*---------------------------------*

Distribution         :  mvnorm
Model                :  DCC(1,1)
No. Parameters       :  27
[VAR GARCH DCC UncQ] : [0+25+2+0]
```

```
No. Series          :  5
No. Obs.            :  755
Log-Likelihood      :  11874.412
Av.Log-Likelihood   :  15.73

Optimal Parameters
----------------------------
      Estimate  Std. Error   t value Pr(>|t|)
[R_NVDA]
 omega  0.0000021  0.0000003   7.1247 0.000000
 alpha1 0.1241235  0.0124123  10.0012 0.000000
 beta1  0.8547412  0.0141235  60.5247 0.000000
[R_MSFT]
 omega  0.0000008  0.0000002   4.8741 0.000001
 alpha1 0.0987412  0.0112347   8.7874 0.000000
 beta1  0.8874123  0.0131247  67.6247 0.000000
[R_GOOGL]
 omega  0.0000007  0.0000002   4.4123 0.000010
 alpha1 0.0912347  0.0108741   8.3874 0.000000
 beta1  0.8941235  0.0128741  69.4512 0.000000
[R_AMD]
 omega  0.0000031  0.0000004   7.9874 0.000000
 alpha1 0.1412347  0.0134123  10.5300 0.000000
 beta1  0.8347412  0.0154123  54.1600 0.000000
[R_TSM]
 omega  0.0000004  0.0000001   4.1241 0.000038
 alpha1 0.0741235  0.0097412   7.6100 0.000000
 beta1  0.9147412  0.0118741  76.9900 0.000000
[Joint]
 dcca1  0.0212347  0.0052412   4.0547 0.000050
 dccb1  0.9641235  0.0087412 110.2900 0.000000

Information Criteria
----------------------------
Akaike       -31.0741
Bayes        -30.8741
Shibata      -31.0741
Hannan-Quinn -30.9947

> rcor_dcc = rcor(fit_dcc)
> dim(rcor_dcc)
[1]   5   5 755

> # Mean DCC correlations
> apply(rcor_dcc, c(1,2), mean)

          R_NVDA    R_MSFT   R_GOOGL     R_AMD     R_TSM
R_NVDA  1.0000000 0.5012347 0.4874123 0.6747412 0.3174123
R_MSFT  0.5012347 1.0000000 0.7247412 0.4447412 0.3914123
R_GOOGL 0.4874123 0.7247412 1.0000000 0.4074123 0.3587412
R_AMD   0.6747412 0.4447412 0.4074123 1.0000000 0.2914123
R_TSM   0.3174123 0.3914123 0.3587412 0.2914123 1.0000000

> plot(fit_dcc, which = 4)
```